

BuMoChi ユーザーガイド

BuMoChi

BuMoChi ユーザーガイド

これは現在の Bmc インターフェースに対応するユーザーガイドの草稿です。BuMoChi ライブラリの目的、連携するアプリケーション、アニメーションクリップの録画、再生、合成など、基本操作を行うコマンド例を説明します。

インストールと入門については、[インストール](#)および[はじめに](#)を参照してください。

1. BuMoChi とは

BuMoChi は、モーションキャプチャーデータとアニメーションキャラクターを使ったパフォーマンスのための SuperCollider ライブラリです。ライブコーダーは動きを素材として扱えます。カメラから受信し、録画し、再生し、タイミングを変更し、選択した身体部位を組み合わせ、その結果をアニメーションキャラクターへ送信できます。BuMoChi は、それぞれ独自のモーションキャプチャートラッキング、コンピューター、音響生成、アニメーションシステムを持つ複数のパフォーマンス用ワークステーション間のライブ共同作業にも対応します。モーションキャプチャーデータを OSCGroups 経由の OSC として共有することで、離れた場所からネットワーク越しに一緒にリハーサルや上演を行えます。

このライブラリは、ネットワークダンス、演劇、音楽、アニメーションなど、動きにライブで介入できる必要があるパフォーマンス状況を対象としています。ある場所でパフォーマーをキャプチャーしながら、別の場所の共同作業者が動きを処理したり、アバターをレンダリングしたりできます。

主なユーザーインターフェースは **Bmc** クラスです。よく使う操作は、ライブコーディングに適するよう意図的に短く設計されています。

```
Bmc.record(\entrance);  
Bmc.stopRecording;  
Bmc.play(\entrance);  
Bmc.rate(0.5);  
Bmc.loop(true);
```

これらのコマンドを使う前に内部クラスを理解する必要はありません。より詳細な制御が必要になったときは、このガイドの後半で説明する補助クラスを利用できます。

エンドユーザーができること

現在の Bmc インターフェースでは、次の操作ができます。

- XR Animator からライブの骨格モーションを受信する。
- ソース名とアバター名によってモーションをルーティングする。
- VMC 対応の Godot キャラクターにモーションを表示する。
- モーションを名前付きクリップとして録画する。
- クリップの一覧表示、選択、名前変更、保存、読み込み、削除を行う。
- クリップの再生、一時停止、シーク、ループ、速度変更を行う。
- 録画間で選択したボーンまたは身体領域をコピーする。
- 異なるモーションソースからライブ合成アバターを構築する。
- フレームが到着しているか、拒否されているかを確認する。

BuMoChi は SuperCollider 内で動作しますが、モーション機能を使うために SuperCollider のオーディオサーバーを起動する必要はありません。パフォーマンスで音声を合成する場合や、Synth のコントロールバスを使って動きを変更する場合に、オーディオサーバーが必要になります。

2. BuMoChi が接続するソフトウェア

完全なネットワークパイプラインは次のとおりです。

XR Animator

→ Bunraku0SCEncoder.py

→ ローカルの SuperCollider + BuMoChi、および OSCGroups

OSCGroups

→ 各共同作業者の SuperCollider + BuMoChi

各ローカル BuMoChi

→ ローカル Bunraku0SCDecoder.py

→ ローカル Godot

1 台のコンピューターだけで使用する場合、最初は OSCGroups を省略できます。

XR Animator → encoder → BuMoChi → decoder → Godot

各コンポーネントの役割は異なります。XR Animator は人物を観察し、エンコーダーは 1 つの完全な、ルート情報を持たない骨格フレームを構成します。エンコーダーは同一のコピーをローカル BuMoChi と OSCGroups に送信します。OSCGroups はこれらのソースフレームを他のワークステーションへ配信し、そこで各フレームは同様に BuMoChi へ入ります。各ローカル BuMoChi は利用可能なすべてのソースを録画、合成、変換して、フィギュアとアバターへ割り当て、完成したシーンだけを自身のデコーダーと Godot レンダラーへ送信します。

分散ソース、ローカル合成

BuMoChi は「分散ソース、ローカル合成」モデルを使用します。共同作業者間を移動するのは、完成したレンダリング済みアニメーションではなく、モーションのソース素材です。各ワークステーションは、自身のモーションソースを直接受信し、遠隔のモーションソースを OSCGroups 経由で受信します。ローカルの SuperCollider/BuMoChi プロセスは、他のワークステーションと同じセッション定義を使い、それらの入力から完全なアニメーションシーンを作成します。最後に、完成したアバターをローカルデコーダー経由でローカル Godot シーンへルーティングします。

この分離は基本原則です。OSCGroups は共有ソースデータ層、BuMoChi はローカルアニメーション合成層、Godot はローカルレンダラーです。Bmc が処理したルート付きフレームが OSCGroups に戻ることはありません。共同作業者が同じセッションと Godot シーンを読み込むと、それぞれが同じ定義済みパフォーマンスを独立して合成し、レンダリングします。これは sc-hacks-redux が制御ソースを共有しながら、各ワークステーションで音響をローカル合成した方法と同じです。

SuperCollider と BuMoChi

SuperCollider は BuMoChi が動作するライブコーディング環境です。公式サイトから SuperCollider をダウンロードします。

- <https://supercollider.github.io/downloads>

BuMoChi リポジトリにはライブラリ本体が含まれています。BuMoChi リポジトリフォルダーを SuperCollider のユーザー拡張ディレクトリ内に置くか、そこからリンクしてください。ディレクトリを確認するには、次を評価します。

Platform.userExtensionDir;

BuMoChi をインストールまたは更新した後は、SuperCollider のクラスライブラリを再コンパイルします。SuperCollider IDE で **Language → Recompile Class Library** を選択してください。

公開インターフェースの現在のソースファイルは次の場所にあります。

Classes/Bmc/

XR Animator

XR Animator は Web カメラを使ってパフォーマーを追跡し、ヒューマノイドモデルをアニメーションさせます。このパイプラインでは、標準 VMC モーションメッセージのソースです。

- プロジェクトとドキュメント：<https://github.com/ButzYung/SystemAnimatorOnline>
- ネイティブアプリケーションのリリース：<https://github.com/ButzYung/SystemAnimatorOnline/releases>
- オンライン版：https://sao.animetheme.com/XR_Animator.html

このパイプラインでは、VMC 出力がネイティブアプリケーションの機能であるため、ネイティブ Electron アプリケーションを使用します。VMC の送信先ホストとポートをエンコーダーに合わせて設定してください。以下のローカル Hello World 例では **127.0.0.1:39537** を使用します。

Python エンコーダーとデコーダー

ここで説明する 2 つの Python コマンドラインスクリプトは、**BuMoChi セットアップ**で説明している 1 コマンドのシェルランチャーを使って同時に起動できます。

必要な Python コマンドラインスクリプトは BuMoChi 配布物に含まれています。

PipelineApplications/BunrakuOSCEncoder.py

PipelineApplications/BunrakuOSCDecoder.py

補助 Python モジュールも同じディレクトリにあります。これらのファイルは一緒に保持してください。スクリプトは Python 3 を使用し、サードパーティ製 Python パッケージを必要としません。

BunrakuOSCEncoder は XR Animator から VMC を受信します。必要な 21 個のヒューマノイドボーンを集め、ルート情報を持たない各 **/bunraku/vmc/frame** を、**57130** のローカル Bmc と **22244** のローカル **OscGroupClient** の両方に送信します。遠隔クライアントは、ルート情報を持たないフレームを **57130** の Bmc に渡します。Bmc は完全なローカルシーンを合成し、最終的なアバタールートを追加して、ルート付き出力を **39538** のローカルデコーダーだけに送信します。

BunrakuOSCDecoder は逆変換を行います。SuperCollider から Bunraku フレームを受信し、Godot 用の標準 VMC メッセージを出力します。

さらに一般的な、VMC パケットを保持するブリッジも次の場所に含まれています。

HelperAppsAndExamples/VMC_Converter_Scripts/

ここで説明する Bmc クラスは、現在 **PipelineApplications** にある固定 Bunraku Frame protocol-v1 エンコーダーとデコーダーを使用します。

OSCGroups

OSCGroups は共同作業で OSC メッセージを運びます。すべての参加者が同じ OSCGroups サーバーとグループに接続します。各参加者はローカル送信ポートとローカル受信ポートを使用し、アプリケーションは通常の UDP OSC を localhost に送信し続けます。

BuMoChi には OSCGroups のドキュメントといくつかのビルド済みクライアントが次の場所に含まれています。

HelperAppsAndExamples/OSCGroups/

上流ソースとビルド手順は次から入手できます。

- <https://github.com/RossBencina/oscggroups>
- <https://github.com/RossBencina/oscpack>
- <http://www.rossbencina.com/code/oscggroups>

最初のローカルテストでは OSCGroups は不要です。1 台のコンピュータでエンコーダーから BuMoChi、デコーダーへの経路が動作した後に追加してください。段階的な詳しいテストは **PipelineApplications/Communication_Tests/** にあります。

OscGroupClient アプリケーションも、**BuMoChi セットアップ**で説明している同じ 1 コマンドのシェルランチャーを使って、必要に応じて起動できます。

Godot

Godot はアニメーションキャラクターをレンダリングします。公式サイトから互換性のある Godot 4 エディターをダウンロードします。

- <https://godotengine.org/download/>

VMC 対応の参照プロジェクトが次の場所に含まれています。

PipelineApplications/GodotVMCReference/project.godot

Godot Project Manager からこのファイルをインポートし、プロジェクトを実行します。参照テスト構成は、再構成された VMC を UDP ポート **39539** で待ち受けます。

後で参照キャラクターまたはプロジェクトを置き換えることができます。新しい Godot プロジェクトが標準 VMC を受け付け、ヒューマノイド骨格が正しくマッピングされていれば、エンコーダー、BuMoChi、デコーダーを変更する必要はありません。

3. Hello World : 動きをキャプチャー、録画、再生する

この最初の例では、すべてを 1 台のコンピュータで実行し、OSCGroups は意図的に省略します。最小限で実用的な BuMoChi セッションを示します。

ポート一覧

送信元	送信先	UDP ポート
XR Animator	Python エンコーダー	39537
Python エンコーダー	BuMoChi	57130
BuMoChi	Python デコーダー	39538
Python デコーダー	Godot	39539

1 つの UDP ポートで待ち受けられるプログラムは 1 つだけです。この例を開始する前に、以前のテストプロセスを停止してください。

手順 1 : Godot を起動する

次をインポートして実行します。

PipelineApplications/GodotVMCReference/project.godot

手順 2 : デコーダーを起動する

PipelineApplications でターミナルを開き、次を実行します。

```
python3 BunrakuOSCDecoder.py \  
--listen-port 39538 \  
--accept-avatar "BunrakuTestAvatar" \  
--verbose
```

手順 3 : SuperCollider で BuMoChi を準備する

次のブロックを評価します。

```
(  
Bmc.reset;  
  
// この名前はエンコーダーの --avatar 値と一致する必要があります。  
Bmc.addAvatar(\BunrakuTestAvatar, "BunrakuTestAvatar");  
Bmc.selectAvatar(\BunrakuTestAvatar);  
  
// このアバターをローカルデコーダー経由で Godot ヘルパーティングします。  
Bmc.avatar(\BunrakuTestAvatar).vmcPort_(39539);  
  
// Python エンコーダーから Bunraku フレームを受信します。  
Bmc.start(57130);  
)
```

状態を確認します。

```
Bmc.status;
```

ポストされるイベントには `running: true` と `port: 57130` が含まれます。

手順 4：エンコーダーを起動する

BuMoChi セットアップに、エンコーダーを最も簡単に起動する方法があります。
同じく PipelineApplications で 2 つ目のターミナルを開き、次を実行します。

```
python3 BunrakuOscEncoder.py \  
  --no-oscggroups \  
  --avatar "BunrakuTestAvatar" \  
  --source "xr-animator" \  
  --verbose
```

手順 5：XR Animator を起動する

XR Animator で VMC の送信先を次のように設定します。

```
Host: 127.0.0.1  
Port: 39537
```

VMC 出力を有効にし、カメラの前で動きます。Godot のキャラクターが動きに追従するはずですが、もう一度 **Bmc.status** を評価し、**received** カウントが増加していることを確認します。

手順 6：クリップを録画する

SuperCollider で録画を開始します。

```
Bmc.record(\hello, "BunrakuTestAvatar", "xr-animator");
```

数秒間動いた後、停止します。

```
~helloClip = Bmc.stopRecording;
```

結果を確認します。

```
~helloClip.size;  
~helloClip.duration;  
Bmc.listClips;
```

手順 7：クリップを再生する

XR Animator の出力を無効にするか、その場で静止して、次を評価します。

```
Bmc.play(\hello);
```

再生設定を変更してみます。

```
Bmc.rate(0.5);      // 次の再生を半分の速度にする  
Bmc.loop(true);  
Bmc.play(\hello);
```

```
Bmc.pause;  
Bmc.resume;  
Bmc.stopPlayback;
```

手順 8：終了する

```
Bmc.stop;
```

その後、XR Animator の出力を無効にし、エンコーダーとデコーダーのターミナルで **Control-C** を押して、Godot プロジェクトを停止します。**Bmc.stop** は BuMoChi の受信と再生を停止しますが、外部アプリケーションは停止しません。

4. Bmc メソッドリファレンス

通常のセッションでは **Bmc** を入口として使用することを推奨します。以下のメソッドは、適切な補助オブジェクトへ処理を委譲します。

システム制御

SuperCollider のクラスライブラリがコンパイルされると、Bmc は入力ポート **57130** でルート情報を持たない Bunraku フレームを自動的に待ち受けます。選択済みデフォルトアバターは **Ishidomaru** です。この識別子に対する完成フレームは **Ishidomaru** という名前に書き換えられ、Godot VMC の送信先ポート **39539** が割り当てられ、ポート **39538** のローカル Bunraku デコーダーへ転送されます。デコーダーへの転送は有効です。この準備済み状態により、エンコーダーが `--avatar "Ishidomaru"` を使用し、Godot が **39539** で待ち受けていれば、個別の初期化ブロックなしですぐにライブ監視と **Bmc.record** を利用できます。

1. **Bmc.start(port: 57130)**

Bunraku Frame OSC レシーバーを起動または再起動します。クラスライブラリのコンパイル後、レシーバーは **57130** で自動起動します。**Bmc.stop** の後に再起動する場合や、別の入力ポートを選択する場合にこのメソッドを呼び出します。ポートはローカルエンコーダーの出力と **OscGroupClient.localRxPort** の両方に一致させる必要があります。

2. **Bmc.stop**

クリップ再生を停止し、未完了の録画をキャンセルして、BuMoChi OSC レシーバーを閉じます。XR Animator、Python、OSCGroups、Godot は停止しません。

3. **Bmc.status**

レシーバー、録画、再生、クリップ、ワイヤーの統計を含むイベントをポストし、返します。主なキーには **running**、**port**、**received**、**rejected**、**dropped**、**recording**、**playing**、**currentClip**、**clipCount** があります。

4. **Bmc.help**

XR-Animator、**BunrakuOSCEncoder**、SuperCollider/Bmc、**BunrakuOSCDecoder**、および現在 Godot VMC 出力ポートを持つ各アバターについて、簡潔なポート設定概要をポストし、返します。

Bmc.help;

対になったエンコーダー／SuperCollider 入力行は、ディスパッチャーの現在のポートを反映します。対になった SuperCollider 出力／デコーダー入力行は **Bmc.decoderPort** を反映します。アバター出力の組は現在の **BmcAvatar.vmcPort** 設定を反映するため、実行時に設定を変更すると表示内容も更新されます。

5. **Bmc.showDispatcherStatus(updateInterval: 0.25)**

OSC/VMC 入力モニターウィンドウを開きます。静的フィールドには監視対象の OSC アドレスとディスパッチャーに設定された UDP ポートが表示されます。動的フィールドには最新の **BmcDispatcher.status** 辞書が表示され、デフォルトでは **0.25** 秒ごとに更新されます。更新間隔を変更するには秒単位の値を指定します。ウィンドウを閉じると更新ルーチンが停止します。

```
Bmc.showDispatcherStatus;          // 1 秒間に 4 回更新
Bmc.showDispatcherStatus(1.0);    // 1 秒間に 1 回更新
```

6. **Bmc.reset**

現在のセッションを停止し、作業中のクリップライブラリ、アバター、レコーダー、プレイヤー、ディスパッチャー、ワイヤーを新しいオブジェクトへ置き換え、Ishidomaru / **39539** のデフォルトルートを復元して、**57130** でレシーバーを再起動します。未保存のメモリー内クリップは失われるため、意図を持って使用してください。

アバターと出力

1. **Bmc.addAvatar(name, displayName)**

アバターの送信先を作成します。入力ストリームを直接受信させる場合、その名前は受信フレーム内のアバター名と一致させる必要があります。

2. **Bmc.avatar(name)**

登録済みの **BmcAvatar** を返します。引数なしの場合、デフォルトアバター **Ishidomaru** を返します。

3. **Bmc.selectAvatar(name)**

以降の再生と完成フレーム録画の送信先としてアバターを選択します。

4. **Bmc.output(destination)**

選択したアバターのデフォルトのローカルデコーダー出力関数を、カスタム送信先へ置き換えます。これは高度な用途のための迂回手段です。

```
Bmc.output(NetAddr("127.0.0.1", 39538));
```

確認またはカスタム処理のための出力として関数を使用することもできます。

5. **Bmc.decoderPort_(port)**

デフォルト出力を使用するすべてのアバターについて、ローカルデコーダーの送信先を設定します。デフォルト：**39538**。

6. **Bmc.forwardDecoder_(flag)**

ローカルデコーダーへの転送を有効または無効にします。デフォルト：**true**。

これらは Bmc が持つ唯一のクラス全体のネットワーク出力制御です。OSCGroups へのソース配信は Bmc ではなく [BunrakuOSCEncoder](#) が担当します。

7. **Bmc.sendCalibrationFrame(port)**

選択したアバターについて、近似的な直立ポーズをローカルデコーダー入力へ1つ送信します。**port** を省略すると、Bmc は現在設定されているデコーダーポートを使用します。デフォルトは **39538** です。ルート付きフレームには、選択したアバターの Godot VMC 送信先も別途埋め込まれます。Ishidomaru のデフォルトは **39539** です。

```
Bmc.sendCalibrationFrame;           // デフォルトのデコーダー入力 39538  
Bmc.sendCalibrationFrame(39548); // 別のデコーダー入力を明示
```

これはパイプライン接続テストであり、特定の VRM モデル用に較正された参照ポーズではありません。

録画

1. **Bmc.record(name, avatar, source, capturePoint, metadata)**

録画を開始します。すべての引数は任意です。SCD がデフォルトの録画形式です。**Bmc.stopRecording** を呼び出すと、完成したクリップはメモリー内に保持され、**BmcClipLibrary.defaultDirectory** に **name.scd** として自動保存されます。

```
Bmc.record;           // すべての入力フレームを録画  
Bmc.record(\take1);   // すべてのフレームを \take1 として録画  
Bmc.record(\take1, "actor", "camA");
```

capturePoint のデフォルトは **\rawFrame** です。参照ポーズによる補完とライブワイヤリングを適用した後の選択アバターを録画するには、**\completedFrame** を使用します。

2. **Bmc.recordScd(name, avatar, source, capturePoint, metadata)**

デフォルトの **Bmc.record** 動作を明示する別名です。

3. **Bmc.recordBmc(name, avatar, source, capturePoint, metadata)**

停止時に従来の **.bmc** アーカイブ形式で保存される録画を開始します。この形式が特に必要な場合にだけ使用してください。

4. **Bmc.stopRecording**

録画を停止し、**BmcMocapClip** を返してメモリー内クリップライブラリへ追加し、現在のクリップとして選択して、録画開始時に選んだ形式でディスクへ保存します。通常の **Bmc.record** と **Bmc.recordScd** は **.scd** を保存し、**Bmc.recordBmc** は **.bmc** を保存します。

5. **Bmc.cancelRecording**

録画を停止し、そのテイクで収集したフレームを破棄します。

6. **Bmc.isRecording**

true または **false** を返します。

クリップライブラリ

1. **Bmc.clips**

メモリー内にあるすべての名前付きクリップの辞書を返します。

2. **Bmc.clip(name)**

名前付きクリップを返します。**nil** の場合は現在のクリップを返します。

3. **Bmc.selectClip(name)**

名前付きクリップを現在のクリップとして選択し、そのクリップを返します。

4. **Bmc.currentClip**

現在選択されているクリップを返します。

5. **Bmc.listClips**

SuperCollider のポストウィンドウに、クリップ名、フレーム数、長さを表示します。現在のクリップには ***** が付きます。

6. **Bmc.showClips**

クリップウィンドウを開きます。最初は、現在メモリーに読み込まれているクリップが表示されます。一覧上部のボタンから、ディスクと再生に関する 2 つの操作を行えます。

- **List saved** は **BmcClipLibrary.defaultDirectory** 内の **.scd** と **.bmc** ファイルを走査し、内容をメモリーに読み込まずに名前を表示します。
- **Play selected** は、選択した保存済みクリップを必要に応じて読み込み、再生を開始します。すでにメモリー内にあるクリップはそのまま再生されます。

メモリー内クリップの行を選択すると、そのクリップが現在のクリップにもなります。**List saved** によって表示された保存済みクリップは、**Play selected** を押すまで読み込まれません。

7. **Bmc.renameClip(oldName, newName)**

メモリー内ライブラリにあるクリップの名前を変更します。

8. **Bmc.removeClip(name)**

メモリーからクリップを削除します。別途保存されたファイルは削除しません。

9. **Bmc.saveClip(name, path) / Bmc.save(name, path)**

クリップアーカイブを書き出します。パスを省略すると、BuMoChi は **Platform.userAppSupportDir** 内の **BmcClips** ディレクトリと **.bmc** 拡張子を使用します。

10. **Bmc.loadClip(path, name) / Bmc.load(path, name)**

保存済みの **.bmc** または **.scd** クリップを読み込みます。拡張子によって読み込み方法が選択されます。**name** を省略すると、ファイル名がクリップ名になります。

11. **Bmc.saveClipScd(name, path)**

メモリー内クリップを、完全で人間が読めるタイムスタンプ／メッセージ形式として明示的に保存または再保存します。**path** を省略すると、デフォルトの **BmcClips** ディレクトリに **name.scd** として保存されます。通常の **Bmc.record** は、**Bmc.stopRecording** の呼び出し時にすでにこの保存を自動実行します。明示的なパスが必要な場合や、既存のメモリー内クリップをもう一度書き出す場合に **Bmc.saveClipScd** を使用します。

```
Bmc.record(\take1);  
// 動きを実行する  
Bmc.stopRecording;  
// BmcClipLibrary.defaultDirectory に take1.scd が作成される
```

12. **Bmc.loadClipScd(path, name)**

読み取り可能な **.scd** クリップを明示的に読み込みます。最初に保存されたタイムスタンプはゼロに正規化され、すべてのフレーム間隔は保持されます。メッセージ行は SuperCollider コードとして解釈されるため、信頼できる **.scd** ファイルだけを読み込んでください。

```
Bmc.loadClipScd(
    BmcClipLibrary.defaultDirectory +/+ "take1.scd",
    \take1
);
```

13. **Bmc.clipToScd(name) / Bmc.convertClipToScd(name)**

録画済みクリップを、人間が読める SuperCollider **.scd** ファイルとして書き出します。ファイルはクリップの **.bmc** アーカイブと同じ場所に置かれ、同じベース名を使用します。たとえば **take1.bmc** は **take1.scd** になります。返り値は書き出したファイルの完全なパスです。

```
~scdPath = Bmc.clipToScd(\take1);
~scdPath.postln;
```

名前付きクリップが現在読み込まれていない場合、このメソッドは **BmcClipLibrary.defaultDirectory** で対応する **.bmc** ファイルを自動的に探して読み込みます。カスタムパスから読み込んだ、またはカスタムパスに保存したクリップは、記憶されたそのパスの隣に書き出されません。

各フレームは、OscRecorder 形式のコメント化されたクリップ相対タイムスタンプと、その後に続くメッセージの **sclang** 表現として書き出されます。

```
//:--[0.125]
[ '/bunraku/vmc/frame', 1, 'Avatar', 'source', 2 ]
```

クリップ全体が1つの **.scd** ファイルに保存され、1,000 メッセージ単位には分割されません。再度書き出すと、同じ名前の既存 **.scd** ファイルは置き換えられます。書き出したファイルは **Bmc.loadClip** または **Bmc.loadClipScd** で復元できます。

再生

1. **Bmc.play(name) / Bmc.playClip(name)**

名前付きクリップを再生します。名前を省略すると、現在のクリップを再生します。

2. **Bmc.pause** と **Bmc.resume**

現在のプレイヤータスクを一時停止し、再開します。

3. **Bmc.stopPlayback**

レシーバーや他の Bmc サービスを停止せず、再生だけを停止します。

4. **Bmc.seek(seconds)**

プレイヤーの次のフレーム位置を、指定時刻以前で最も近いフレームへ移動します。

5. **Bmc.rate(value)**

再生速度を設定します。**1.0** は元のタイミング、**0.5** は半分の速度、**2.0** は2倍速です。値はゼロより大きくなければなりません。

6. **Bmc.loop(flag)**

繰り返し再生を有効または無効にします。

録画を合成する

1. **Bmc.combineClips(target, source, bones, result, startIndex)**

選択したボーンを一方のクリップから他方へコピーし、結果を新しい **BmcAnimationClip** として保存します。

```
Bmc.combineClips(
    \baseTake,
    \armTake,
    \leftArm,
    \combined
);
```

組み込み身体グループには **\leftArm**、**\rightArm**、**\arms**、**\leftLeg**、**\rightLeg**、**\legs**、**\torso**、**\upperBody**、**\all** があります。正確なボーン名の配列を指定することもできます。

2. **Bmc.combine(targetFrame, sourceFrame, bones)**

2つの個別 Bunraku フレーム間で、選択したボーンをコピーする低水準操作です。

3. **Bmc.rseq(targetSequence, sourceSequence, bones, startIndex)**

一連のフレーム範囲にわたり、選択したボーンを置き換える低水準シーケンス操作です。

4. **Bmc.bone(frame, boneName)**

フレーム内の1つの名前付きボーンについて、7つのトランスフォーム値を返します。

ライブ合成

1. **Bmc.wire(source, bones, target, sourceAvatar, priority)**

永続的なワイブルーティング規則を作成します。例：

```
~armWire = Bmc.wire(  
    "camera-a",  
    \leftArm,  
    \composite,  
    "performer-a"  
);
```

一致する左腕のトランスフォームが **\composite** アバターへコピーされます。他のボーンには、そのアバターの現在のポーズまたは参照ポーズが保持されます。

2. **Bmc.unwire(wire)**

1つのワイヤーオブジェクトを削除します。

3. **Bmc.listWires**

有効なワイヤーをポストし、返します。

4. **Bmc.clearWires**

すべての Bmc アバターから、すべてのライブワイヤーを削除します。

5. Bmc クラス概要

ほとんどのユーザーは **Bmc** だけで作業できます。以下の補助クラスは、責務の分け方を説明し、高度な作業のための拡張点を提供します。

Bmc

公開ライブコーディングファサードです。現在の作業環境を所有し、短いユーザーコマンドをディスパッチャー、レコーダー、クリップライブラリ、プレイヤー、アバター、ワイヤーの操作へ変換します。また、低水準のフレームおよびシーケンス合成メソッドも保持します。

BmcDispatcher

/bunraku/vmc/frame OSC メッセージを受信して検証し、拒否されたフレームや連続性を失ったフレームを数え、有効なフレームを登録済みの送信先とアバターヘルルーティングします。

BmcAvatar

制御可能な1体の出力キャラクターを表します。現在のポーズとニュートラルな参照ポーズを保持し、不足データを補完し、ライブワイヤーを適用して、完成フレームを関数またはネットワーク送信先へ送ります。

BmcFrame

1つの Bunraku アニメーションフレームを型付きで表現します。プロトコルバージョン、アバター、ソース、フレーム番号、エンコーダーのタイムスタンプ、**BmcPose** を含みます。クラスオブジェクトと OSC メッセージ配列の間を変換します。

BmcPose

骨格の空間状態です。標準化されたボーン名をトランスフォームへ対応付けるコレクションです。選択したボーンをコピーし、別のポーズから不足ボーンを補完できます。

BmcBoneTransform

1つのボーンの位置と回転を、**x, y, z, qx, qy, qz, qw**の7つの値として保持します。

BmcBoneSets

クリップ合成とライブワイヤリングで使用する名前付き身体領域です。**leftArm, rightArm, legs, torso, upperBody**などのグループを提供します。

BmcClip

時間付きフレーム列の基底クラスです。フレームアクセス、相対時刻、長さ、コピー、アーカイブの読み書きを提供します。

BmcMocapClip

モーションキャプチャー入力の録画から生成された **BmcClip** です。フィルター、キャプチャーポイント、パフォーマンス、ソースなどのキャプチャーメタデータを保持できます。

BmcAnimationClip

直接キャプチャーではなく、編集、身体部位の合成、または将来のアルゴリズム生成によって作られた **BmcClip** です。

BmcClipRecorder

検証済みフレームを収集し、アバターとソースでフィルタリングし、到着時刻の相対関係を保持して、停止時に **BmcMocapClip** を返します。

BmcClipPlayer

記録されたタイミングに従ってクリップフレームをスケジュールします。再生、一時停止、再開、停止、シーク、ループ、速度を処理し、フレームをアバター、関数、または **NetAddr** へ送信します。

BmcClipLibrary

名前付きのメモリー内クリップコレクションと現在の選択を管理します。クリップ一覧、クリップ一覧 GUI、名前変更／削除、アーカイブの保存／読み込みを実装します。

BmcWire

選択したソースと身体領域を対象アバターへ接続するライブ規則です。ソース名とソースアバター名でフィルタリングでき、複数のワイヤーが同じ対象へ作用する場合の優先度を持ちます。

BunrakuParser

シンボルでラベル付けされた Bunraku メッセージとコントロールバス配置のための旧互換パーサーです。固定 Bunraku Frame protocol-v1 経路では、主に **BmcFrame**、**BmcPose**、**Bmc.bone** を使用します。

追加テストとトラブルシューティング

リポジトリには4段階の通信テストが含まれています。

1. XR Animator → Godot
2. XR Animator → encoder → OSCGroups → remote BuMoChi
3. SuperCollider playback → decoder → Godot
4. local and remote sources → BuMoChi synthesis → local decoder → local Godot

テストは次の場所にあります。

PipelineApplications/Communication_Tests/

各テストには、ポート一覧、起動順序、合格条件、一般的な障害が記載されています。完全なシステムが動作しない場合はテスト 1 に戻り、コンポーネントを 1 つずつ追加してください。